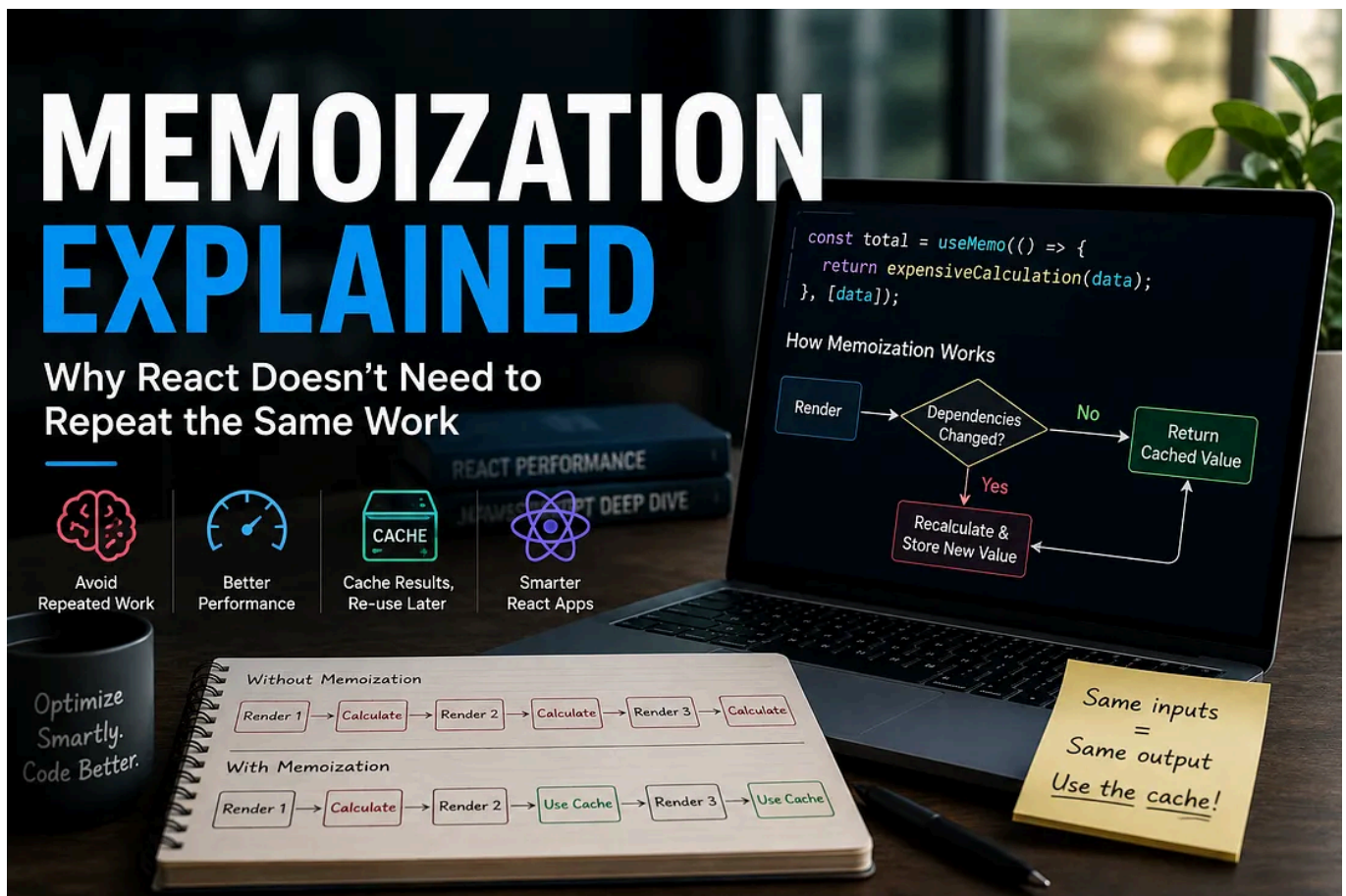




Then I learned `useMemo` in React.

Almost every tutorial explained it like this:

“`useMemo` is used to optimize expensive calculations.”

While that statement is technically correct, it never answered the real question:

Why do we need to optimize them in the first place?

The moment I understood the actual problem, memoization became one of the simplest concepts in React.

Let’s understand it from the beginning.

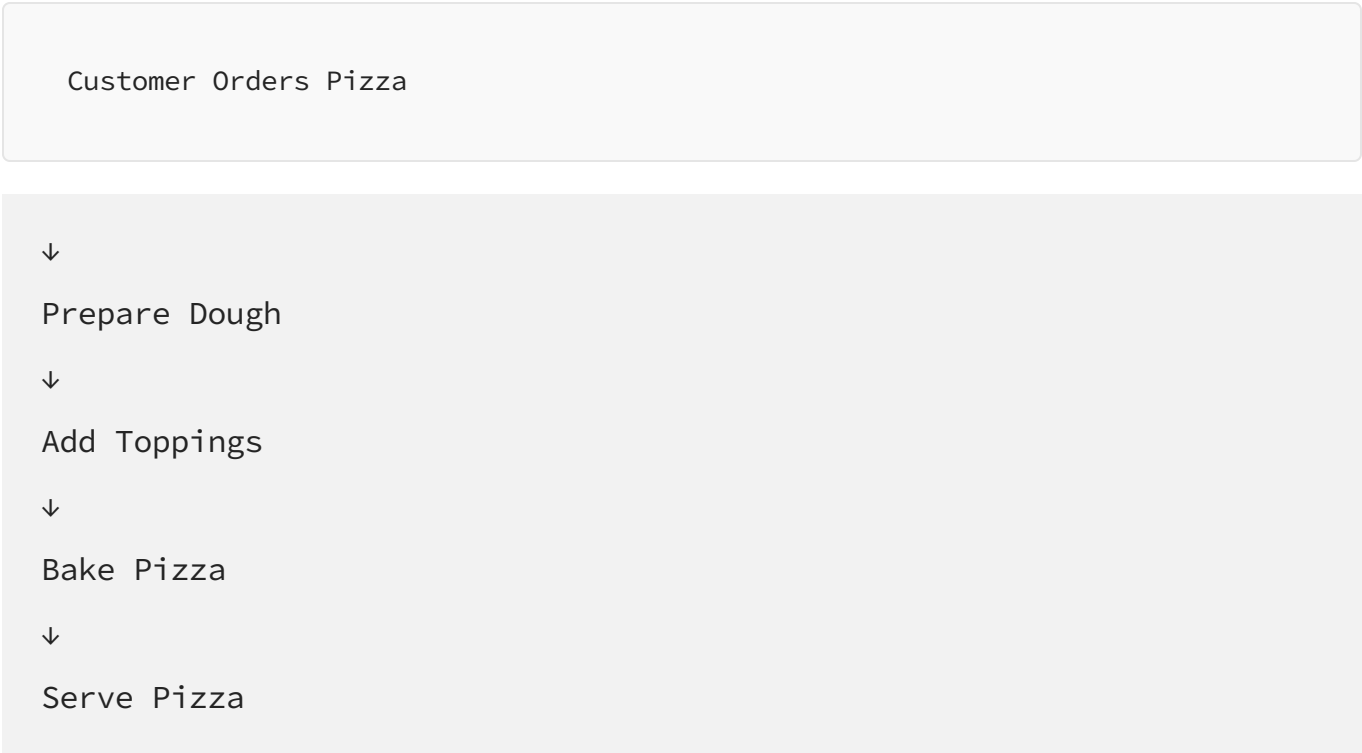
. . .

The Real Problem

Imagine you’re running a pizza restaurant.

A customer orders a **Veg Pizza**.

The chef starts preparing it.



It takes around **15 minutes**.

A few minutes later, the customer says:

“Can I also get a Coke?”

Now ask yourself:

Should the chef prepare the pizza again?

Of course not.

The pizza is already ready.

The only new work is bringing a Coke.

Any smart chef would simply reuse the existing pizza instead of making it again.

. . .

Unfortunately, React Doesn't Think Like That

Whenever a React component re-renders, React executes the **entire component function** again.

For example:

```
function App() {
```

```
  const total = expensiveCalculation();  
  return (  
    <>  
      <button>Toggle Theme</button>  
    </>  
  )  
}
```

```
);  
}
```

Now imagine the user only changes the theme.

Did the data used by `expensiveCalculation()` change?

No.

But React still executes it again because the component rendered again.

The flow looks like this:

Theme Changed

```
↓  
Component Re-rendered  
↓  
expensiveCalculation()  
↓  
Again...  
↓  
Again...  
↓  
Again...
```

React isn't trying to be inefficient.

It simply follows one simple rule:

| **Whenever a component renders, execute the component function again.**

Sometimes that's perfectly fine.

But sometimes we're repeating expensive work even though nothing relevant has changed.

That's the problem memoization solves.

. . .

What is Memoization?

Memoization is an optimization technique where we **store the result of a calculation and reuse it if the inputs haven't changed.**

Instead of doing the same work repeatedly,

we simply reuse the previously computed result.

In simple words,

| **If nothing has changed, don't calculate it again.**

That's memoization.

. . .

Another Real-Life Example

Imagine you're a teacher.

You have marks for **10 lakh students**.

Calculating the average takes several seconds.

Now suppose you only change your website's theme.

Question:

Should the average of all students be calculated again?

Obviously not.

The marks haven't changed.

Only the theme changed.

A smart system would simply reuse the previous average.

Theme Changed

↓

Marks Changed?

↓

No

↓

Reuse Previous Average

Instead of

Theme Changed

↓

Calculate Average Again 🤖

This is exactly how memoization works.

• • •

React's Version of Memoization

React provides the `useMemo` Hook.

```
const totalPrice = useMemo(() => {  
  return expensiveCalculation();  
}, [dependencies]);
```

Whenever the component renders,

React checks the dependency array.

If the dependencies haven't changed,

React returns the previously stored value.

If any dependency changes,

React runs the calculation again and stores the new result.

Internally, the process looks like this:

Component **Re**-render

↓

Dependencies Changed?

↓

Yes → Run Calculation

|

No

↓

Return Cached Value

Notice something important.



A promotional banner for Medium membership. It features a light gray grid background. On the left, a large yellow rectangle contains the text "Find your next rabbit hole." in a black serif font. To the right of this, a blue rectangle contains the text "Medium members get access to everything." in a black sans-serif font. On the far right, a dark gray rounded rectangle contains the text "Upgrade today" in a white sans-serif font. A thick, dark, hand-drawn line starts from the right side of the yellow rectangle and loops around the blue rectangle.

Find your next rabbit hole.

Medium members get access to everything.

Upgrade today

React does **not** cache the function.

It caches **the value returned by that function**.

This is one of the most common misconceptions among beginners.

. . .

Why Doesn't React Memoize Everything Automatically?

You might wonder,

“If memoization is so useful, why doesn't React automatically remember every calculation?”

The answer is simple.

Memoization itself has a cost.

React has to:

- Store the calculated value in memory.
- Remember its dependencies.
- Compare every dependency on each render using `Object.is()`.
- Decide whether to reuse the old value or compute a new one.

If React memoized everything automatically, your application would consume more memory and spend extra time comparing dependencies — even for calculations that are already extremely fast.

Sometimes recalculating a value is actually cheaper than memoizing it.

That's why React leaves the decision to developers.

. . .

When Should You Use Memoization?

Memoization makes sense when all of these conditions are true:

- The calculation is expensive.
- The component re-renders frequently.
- The inputs rarely change.
- Recalculating provides no additional benefit.

Good examples include:

- Filtering thousands of products.
- Sorting large datasets.
- Calculating statistics.
- Processing charts.
- Heavy mathematical operations.
- Complex search algorithms.

These calculations may take noticeable time, so avoiding unnecessary work can significantly improve performance.

. . .

When Should You Avoid It?

This is where many developers make mistakes.

After learning `useMemo`, they start wrapping almost everything inside it.

For example:

```
const fullName = useMemo(() => {  
  return firstName + " " + lastName;  
}, [firstName, lastName]);
```

This calculation is just string concatenation.

It takes almost no time.

Adding memoization here only makes the code longer without providing any real performance benefit.

The same applies to:

- Basic arithmetic
- Small array lookups
- Simple object creation
- Boolean checks

- Small string manipulations

Modern JavaScript engines are incredibly fast at these operations.

Memoizing them is usually unnecessary.

. . .

The Hidden Drawbacks of Unnecessary Memoization

Many developers think:

“More `useMemo` means better performance.

In reality, the opposite can happen.

Using memoization where it isn't needed introduces its own overhead.

1. Increased Memory Usage

Every memoized value is stored until React decides it no longer needs it.

If you're memoizing lots of small values, you're consuming memory for almost no benefit.

. . .

2. Dependency Comparison Cost

On every render, React compares all dependencies.

```
useMemo(() => calculate(), [a, b, c]);
```

Even if the calculation doesn't run,

React still compares `a`, `b`, and `c`.

For small calculations, these comparisons may cost more than recalculating the value itself.

• • •

3. More Complex Code

Compare these two examples.

Simple:

```
const totalProducts = products.length;
```

Memoized:

```
const totalProducts = useMemo(() => {  
  return products.length;  
}, [products]);
```

Both produce exactly the same result.

The second version is longer, introduces another Hook, and doesn't improve performance in any meaningful way.

Clean code is usually better than over-optimized code.

. . .

4. Risk of Stale Values

Another common mistake is forgetting dependencies.

```
const total = useMemo(() => {  
  return price * quantity;  
}, [price]);
```

Since `quantity` is missing, React keeps returning an outdated value.

Incorrect dependencies are one of the most common causes of subtle React bugs.

. . .

5. Optimizing Without Measuring

One of the biggest engineering mistakes is optimizing code that isn't actually slow.

If users don't experience any performance issues, adding `useMemo` everywhere only increases complexity without solving a real problem.

Always optimize based on evidence, not assumptions.

. . .

The Mental Model

Instead of asking yourself:

“Can I use `useMemo` here?”

Ask a better question:

“Is React repeating an expensive calculation even though nothing relevant has changed?”

If the answer is **yes**, memoization is probably the right choice.

If the answer is **no**, let React calculate the value again.

Most of the time, that's perfectly fine.

. . .

Final Thoughts

Memoization isn't a React feature.

It's a general computer science optimization technique that React exposes through the `useMemo` Hook.

Its goal isn't to make your application magically faster.

Its goal is much simpler:

Avoid repeating expensive work when the inputs haven't changed.

Remember these three rules:

- Don't use `useMemo` because you learned it.
- Don't avoid `useMemo` because it looks complicated.
- Use it only when it solves a real performance problem.

The best optimization isn't using more Hooks.

The best optimization is writing simple code first, measuring performance, and only then optimizing the parts that actually need it.

Because in software engineering, **the fastest calculation is often the one you never had to perform twice.**

Happy coding ❤️

Memoization +

JavaScript ✓

Programming ✓

Web Development ✓